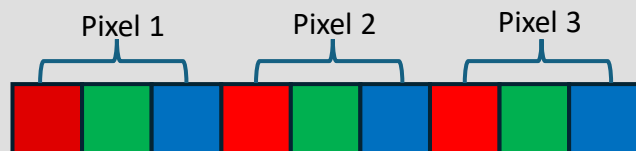


Data Layout Transformations

- N: Number of images in the batch.
- C: Number of Channels per image (e.g., 3 for Red, Green, Blue).
- H: Height of the image in pixels.
- W: Width of the image in pixels.

NHWC



- **Order:** Batch, Height, Width, Channels
- **What it means:** For each pixel, all its color/feature values are stored next to each other.
- **Typical:** TPUs/mobile NPUs; kernels that combine channels at a pixel
- Order in the above image: pixel 1(red)-> pixel 1 (green), pixel 1 (blue), pixel 2(red), pixel 2 (green), pixel 2 (blue), pixel 3(red), pixel 3 (green), pixel 3 (blue)

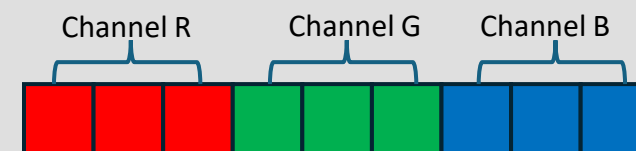
Reorder/transpose axes



Example permutation: [0, 3, 1, 2]
turns NHWC into NCHW

*No data changes—just the order
we store it.*

NCHW



- **Order:** Batch, Channels, Height, Width
- **What it means:** For each channel, all the pixels in the image are stored next to each other.
- **Good when:** You sweep over images one channel at a time.
- **Typical:** Convolution on many GPUs

For large matrix multiplies, we often go beyond NHWC/NCHW and use packed tile layouts that match the GEMM kernel

GEMM stands for General Matrix Multiplication. It's the mathematical name for the core operation that powers deep learning and scientific computing: multiplying two large matrices together.

Hardware-Specific Optimizations - GPU

GPU Optimizations

Objective: Optimize for parallel execution and memory bandwidth efficiency.

Key Considerations:

- GPUs excel at parallel workloads but require **coalesced memory access** (adjacent threads accessing contiguous memory).
- **Tensor Cores** (in modern GPUs) accelerate mixed-precision (FP16/FP32) operations.

Coalesced Memory Access:

- Ensure tensors are stored in memory layouts (e.g., NHWC for images) that allow threads to read contiguous blocks.
- Avoid strides or padding that disrupt memory locality.

Tensor Cores:

- Use mixed precision (FP16 for weights, FP32 for gradients) to leverage hardware acceleration.
- Enabled via frameworks like TensorFlow (tf.keras.mixed_precision).

Example: For a convolutional layer, prefer **channels last** (NHWC) over **channels_first** (NCHW) on NVIDIA GPUs (example in next slide).

Hardware-Specific Optimizations – GPU (cont)

GPU Optimizations

Example: For a convolutional layer, prefer **channels_last (NHWC)** over **channels_first (NCHW)** on NVIDIA GPUs.

In a Tensor with Batch size (N) = 1 (single image), Height (H) = 2, Width (W) = 2, Channels (C) = 3 (RGB)

Then the data for one image (2x2 RGB) is:

Pixel (0,0): [R00, G00, B00]

Pixel (0,1): [R01, G01, B01]

Pixel (1,0): [R10, G10, B10]

Pixel (1,1): [R11, G11, B11]

For a 2D convolution:

- In NHWC, threads processing a 3x3 patch load contiguous pixels (good locality).
- In NCHW, threads fetching the same patch scatter across memory (poor locality) – because channels are stored separately ([all R][all G][all B]).